Chapter 1

Git Basics, Repository, Commits, Branches

Quan Nguyen

August 2026

Table of Contents

1. Git Fundamentals: Version Control, Snapshots, and the Three States
2. Git Repository / Git Commit
3. Git Branches

Table of Contents

1. Git Fundamentals: Version Control, Snapshots, and the Three States
2. Git Repository / Git Commit
3. Git Branches

Session Objectives

- Explain what version control is
- Describe why Git is different from other VCS tools
- Identify the three states a file can be in
- Understand the three sections of a Git project

What Is Version Control

- Records changes to files over time
- Allows recalling specific versions later
- Works for source code, images, layouts, almost any file type

Why Version Control Matters

- Revert a file or whole project to a previous state
- Compare changes across time
- Identify who changed what, and when
- Recover easily from mistakes or lost files

Local Version Control Systems

- Method: copy files into another directory (sometimes timestamped)
- Simple, but error-prone
- Easy to overwrite the wrong file

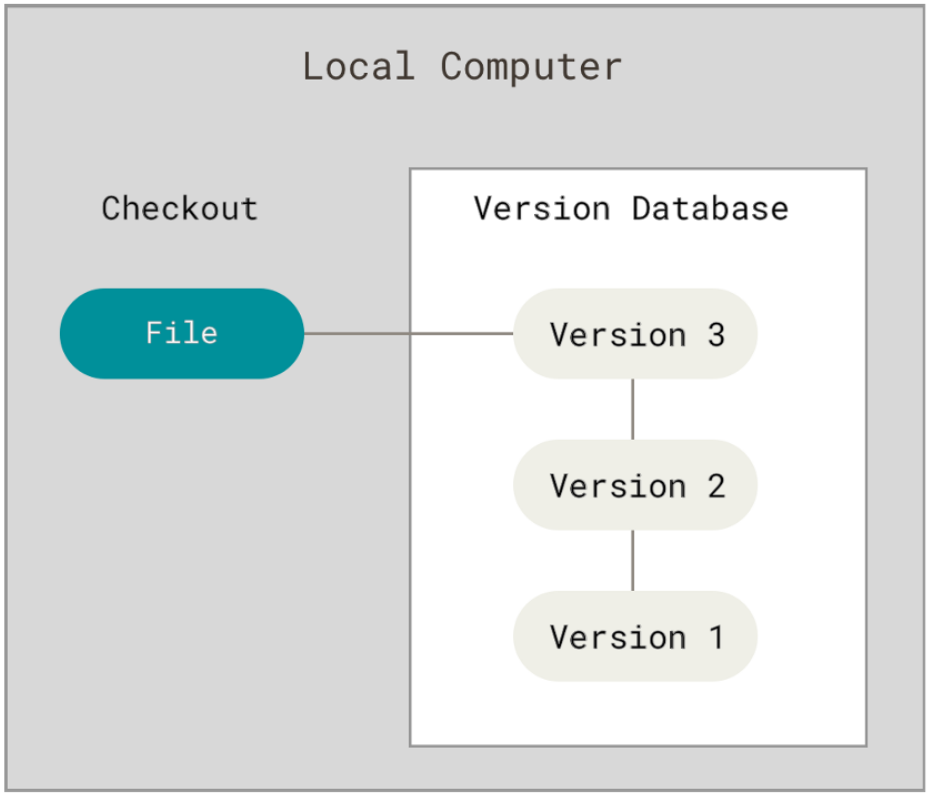


Figure 1. Local version control diagram

Local VCS Tools: RCS

- Simple database tracking file changes
- Stores patch sets (differences between versions)
- Reconstructs any past file state from patches

Centralized Version Control Systems (CVCS)

- Examples: CVS, Subversion, Perforce
- One central server holds all versioned files
- Clients check out files from that server

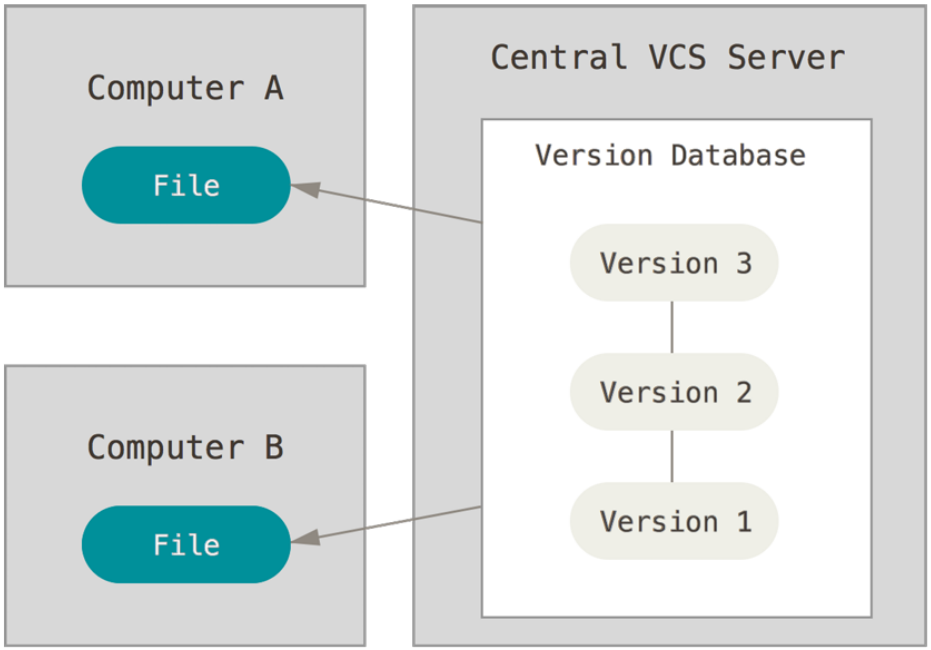


Figure 2. Centralized version control diagram

Advantages of CVCS

- Administrators get fine-grained access control
- Everyone can see what others are doing
- Easier to administer than many local databases

Disadvantages of CVCS

- Single point of failure (the central server)
- Server downtime blocks all collaboration
- Data corruption without backup = total history loss

Distributed Version Control Systems (DVCS)

- Examples: Git, Mercurial, Darcs
- Every client fully mirrors the repository + history
- Every clone = a full backup
- Server failure does not destroy project history

Distributed Version Control Systems (DVCS)

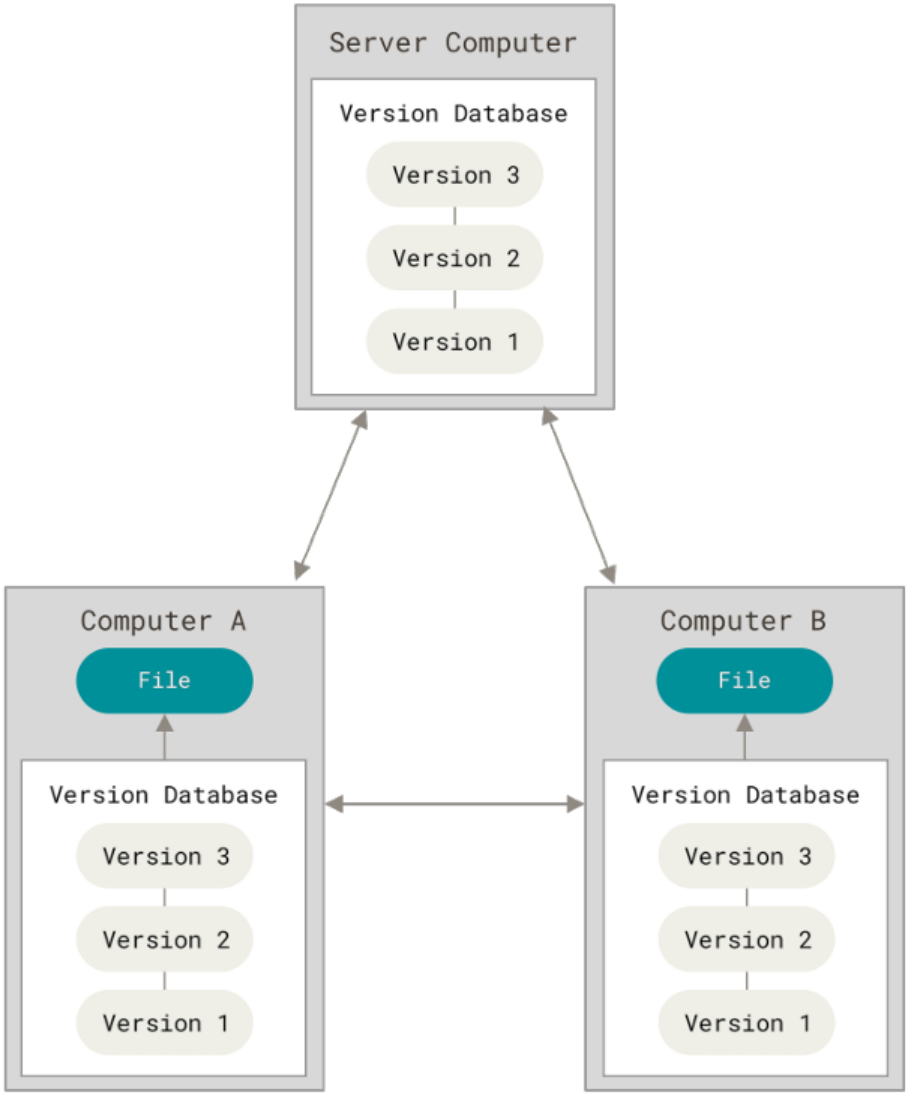


Figure 3. Distributed version control diagram

Distributed Workflow Flexibility

- Can work with multiple remotes at once
- Supports several collaboration models
- Enables hierarchical workflows not possible in CVCS

A Short History of Git

- Originated from Linux kernel development
- 1991–2002: changes shared as patches and archives
- 2002: kernel project adopts BitKeeper (proprietary DVCS)

Why Git Was Created

- 2005: BitKeeper relationship breaks down
- Linus Torvalds and community build their own tool
- Design goals: speed, simplicity, non-linear development, fully distributed, handles large projects

What Git Achieved

- Fast performance
- Efficient with large projects
- Powerful branching system for non-linear development

Snapshots, Not Differences

- Most VCS tools store data as file-based changes (diffs)
- Git does not work this way

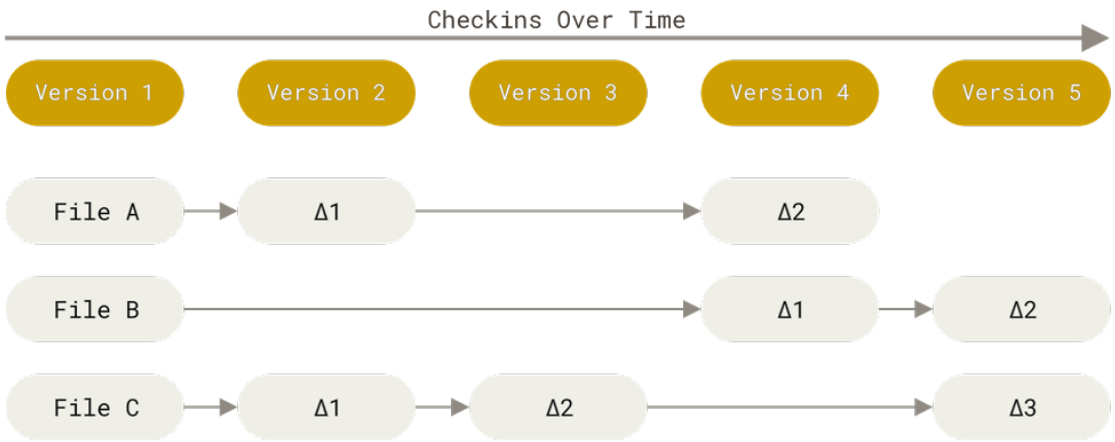


Figure 4. Storing data as changes to a base version of each file

How Git Stores Data

- Git = series of snapshots of a mini filesystem
- Every commit = a picture of all tracked files at that moment
- Stores a reference to that snapshot

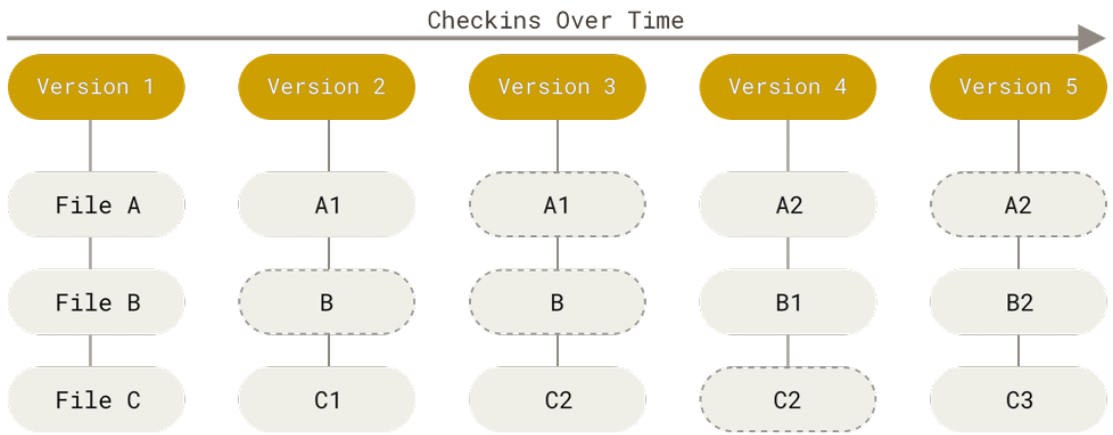


Figure 5. Storing data as snapshots of the project over time

Efficiency Through Snapshot Linking

- Unchanged files are not re-stored
- Git links to the previous identical file
- Data model = stream of snapshots

Why This Matters for Branching

- Core distinction from most other VCS tools
- Reshapes how version control works
- Foundation for Git's branching model

Nearly Every Operation Is Local

- Most operations need no network access
- Entire project history lives on local disk
- Browsing history feels instantaneous

Working Offline With Git

- History reads are local, not remote
- Comparing old vs. current file = local calculation
- Can commit work offline, push later

Contrast With Centralized Tools

- Perforce: little can be done offline
- Subversion/CVS: can edit offline, cannot commit offline
- Git: full workflow available offline

Git Has Integrity

- Everything is checksummed before storage
- Referred to by checksum afterward
- Impossible to change content without Git noticing

The SHA-1 Hash Mechanism

- 40-character hexadecimal string
- Example hello.txt: "Hello" $\rightarrow$ SHA-1 hash function $\rightarrow$ 24b9da6552252987aa493b52f8696cd6d3b00373 (40 hexadecimal characters)
- Referred to by checksum afterward
- Impossible to change content without Git noticing. For example: change to Hello! will result in a different hash

The Three States of a File

- Modified — changed, not yet committed
- Staged — marked for the next commit
- Committed — safely stored in local database

The Three Sections of a Git Project

- Working tree
- Staging area
- Git directory

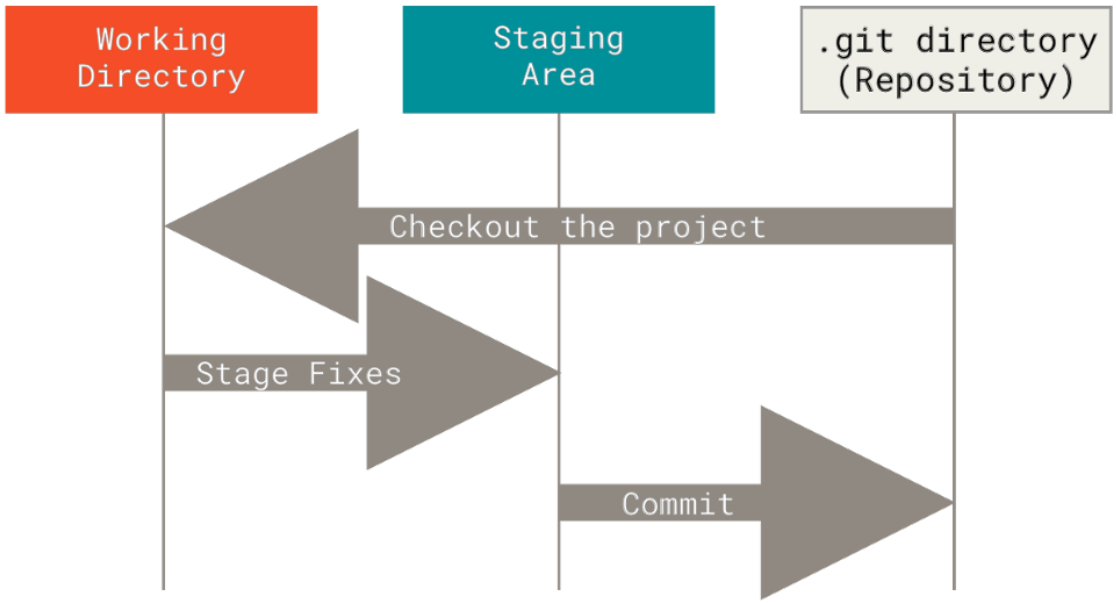


Figure 6. Working tree, staging area, and Git directory

The Working Tree

- Single checkout of one project version
- Files pulled from compressed database
- Placed on disk to use/modify

The Staging Area

- File inside the Git directory
- Records what goes into the next commit
- Technical name: "the index"

The Git Directory

- Stores metadata + object database
- Most important part of Git
- Copied when a repository is cloned

The Basic Git Workflow

- Modify files in working tree
- Stage selected changes
- Commit staged snapshot to Git directory

Interpreting File States

- In Git directory $\rightarrow$ committed
- Modified + added to staging $\rightarrow$ staged
- Changed since checkout, not staged $\rightarrow$ modified

Why the Command Line

- Only place to run **all** Git commands
- GUIs implement only partial functionality
- Command-line skill transfers to any GUI

Installing Git

- Available on Linux, macOS, Windows
- Install via package manager, installer, or source
- Update even if already installed

First-Time Git Setup

- One-time setup per machine
- Tool: git config
- Controls how Git looks and operates

First-Time Git Setup

- One-time setup per machine
- Tool: git config
- Controls how Git looks and operates

Configuration Levels

- System level → all users — [path]/etc/gitconfig
- Global level → current user — /.gitconfig or /.config/git/config
- Local level → single repository — .git/config
- Local overrides Global overrides System

Practical Configuration Commands

- git config –global user.name "Name"
- git config –global user.email "email@example.com"
- –global affects all projects of current user
- to override this with a different name or email address for specific projects, run the command without the –global option

Summary of Key Concepts

- Git = distributed VCS, stores snapshots not differences
- Operations are mostly local $\rightarrow$ fast
- Data integrity via SHA-1 checksums
- Every file: modified, staged, or committed

Table of Contents

1 Git Fundamentals: Version Control, Snapshots, and the Three States

2 Git Repository / Git Commit

3 Git Branches

Session Objectives

- Configure and initialize a repository
- Start and stop tracking files
- Stage and commit changes
- Ignore files with patterns
- Undo mistakes
- Browse history and view diffs
- Push and pull from remotes

Getting a Git Repository

- Two ways to obtain a repository:
  1. Turn an existing local directory into one
  2. Clone an existing repository

Initializing a Repository

- Navigate to the project directory
- Run: `git init`
- Creates a `.git` subdirectory (repository skeleton)
- Nothing is tracked yet at this point

Cloning an Existing Repository

- `git clone <url>`
- Copies the entire history, not just the latest snapshot
- Every clone = a full backup of the project

What `git clone` Actually Does

- Creates a new directory
- Runs `git init` inside it
- Adds a remote (default name: `origin`)
- Runs `git fetch` from that remote
- Checks out the latest commit with `git checkout`

Recording Changes — Overview

- Every file is either **tracked** or **untracked**
- Tracked files: unmodified, modified, or staged
- Untracked files: everything else in the working directory

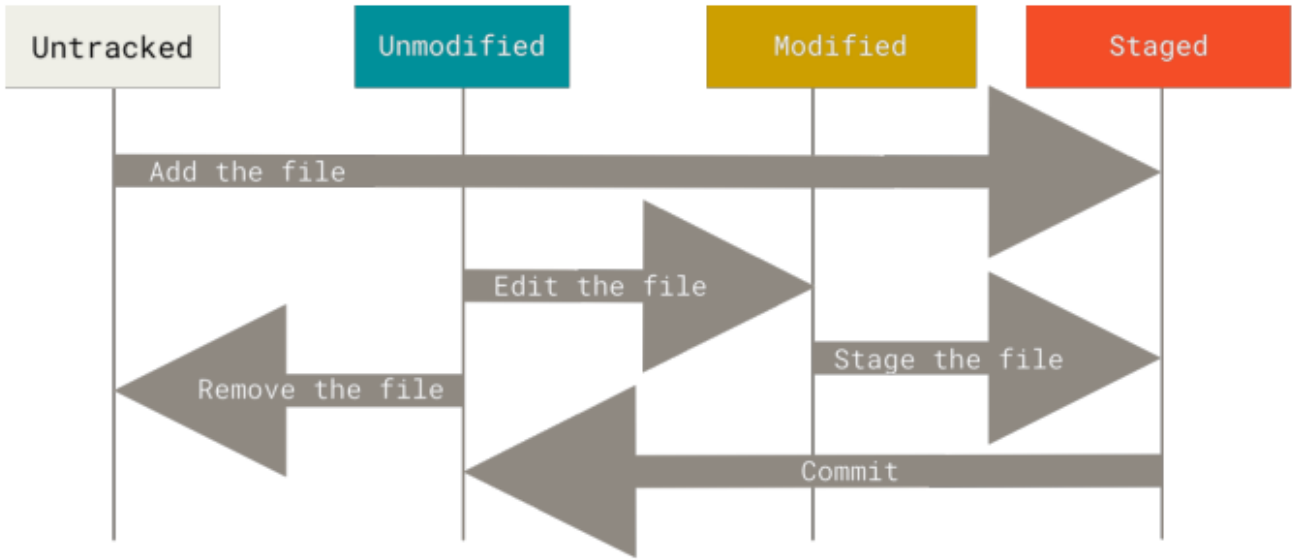


Figure 8. The lifecycle of the status of your files

Checking File Status

- Command: `git status`
- Shows which branch you're on
- Shows staged, unstaged, and untracked files

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working tree clean
$ echo 'My Project'  README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Untracked files:
(use "git add file ..." to include in what will be committed)
README
nothing added to commit but untracked files present (use "git add" to track)
```

Tracking New Files

- Command: `git add <file>`
- Begins tracking a new file
- Also used to stage modified files

$ git add README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
(use "git restore –staged ¡file¿..." to unstage)
new file: README

Staging Modified Files

- Modified tracked files show as “not staged” in `git status`
- `git add` moves them to “staged” for the next commit
- Staging is a snapshot decision, not a save-everything action

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

```
$ git add CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)
```

Short Status Output

- `git status -s` or `git status --short`
- Compact view: two-column flags per file
- Useful for quick checks during a busy session

Ignoring Files: `.gitignore`

- Lists patterns of files Git should not track
- Common for build artifacts, logs, local configs
- Prevents accidental commits of irrelevant/sensitive files

Viewing Staged/Unstaged Changes

- `git diff` $\rightarrow$ changes not yet staged
- `git diff --staged` $\rightarrow$ changes staged for next commit
- Helps confirm exactly what will be committed

Committing Changes

- Command: `git commit`
- Opens an editor for the commit message
- Records the staged snapshot permanently

Inline Commit Messages

- `git commit -m "message"`
- Skips opening the editor
- Good for quick, well-understood changes

Skipping the Staging Area

- `git commit -a`
- Automatically stages all tracked, modified files
- Skips explicit `git add` for tracked files only

Removing Files

- `git rm <file>` — removes from working directory and stages the removal
- `git rm --cached <file>` — stops tracking but keeps the file locally

Moving/Renaming Files

- `git mv old_name new_name`
- Shortcut for: rename, `git rm old_name`, `git add new_name`
- Git tracks renames automatically via similarity detection

Viewing the Commit History

- Command: `git log`
- Lists commits in reverse chronological order
- Default: SHA-1 hash, author, date, message

Useful `git log` Options

- `git log -p` → shows the diff for each commit
- `git log --stat` → shows abbreviated stats per commit
- `git log --oneline` → compact one-line-per-commit view

Formatting and Filtering Log

- `git log --graph` → visualizes branch/merge structure
- `--author`, `--since`, `--until` → filter by author or date range
- `--pretty=format:` → custom output formatting

Undoing Things — Overview

- Mistakes are common and recoverable
- Some undo operations **cannot** be reversed — use with care
- Covers amending commits, unstaging, and reverting local changes

Amending a Commit

- Command: `git commit --amend`
- Replaces the previous commit with a new one
- Useful for fixing a forgotten file or a typo in the message

Unstaging a Staged File

- Command: `git reset HEAD <file>`
- Removes a file from the staging area
- Working directory changes remain untouched

Discarding Local Changes

- Command: `git restore <file>`
- Reverts a file back to the last committed version
- Dangerous: uncommitted changes are permanently lost

Working with Remotes — Overview

- Remote repositories = versions of the project hosted elsewhere
- Enable collaboration across machines and teams
- Managed with the `git remote` command family

Viewing Remotes

- `git remote` → lists remote shortnames
- `git remote -v` → shows URLs for fetch/push
- Default remote name after cloning: `origin`

Adding a Remote

- Command: `git remote add <shortname> <url>`
- Registers a new remote reference
- Enables fetching from and pushing to that project

Fetching from Remotes

- Command: `git fetch <remote>`
- Downloads data not yet in the local repository
- Updates remote-tracking branches; does **not** merge automatically

Pushing to Remotes

- Command: `git push <remote> <branch>`
- Uploads local commits to the remote repository
- Fails if the remote has commits you don't have locally

Inspecting a Remote

- Command: `git remote show <remote>`
- Shows remote URL, tracked branches, push/pull configuration
- Useful for diagnosing sync issues

Removing and Renaming Remotes

- `git remote rename <old> <new>`
- `git remote remove <name>`
- Keeps remote references organized as projects evolve

Tagging — Purpose

- Used to mark specific points in history (e.g., releases)
- Tags do not move like branches
- Two types: lightweight and annotated

Lightweight Tags

- Command: `git tag <tagname>`
- Just a pointer to a specific commit
- No extra metadata stored

Annotated Tags

- Command: `git tag -a <tagname> -m "message"`
- Stores tagger name, email, date, and message
- Recommended for public/release tags

Listing and Sharing Tags

- `git tag` $\rightarrow$ lists all tags
- Tags are **not** pushed by default
- Use `git push <remote> <tag>` to share them

Table of Contents

1 Git Fundamentals: Version Control, Snapshots, and the Three States

2 Git Repository / Git Commit

3 Git Branches

Why Branching Matters

- Branching = diverging from the main line of development
- Work continues without disturbing the main line
- In many VCS tools, branching is expensive (full directory copy)
- In Git, branching is lightweight and nearly instantaneous

Git's "Killer Feature"

- Some call Git's branching model its killer feature
- Branch creation and switching are extremely fast
- Git encourages frequent branching and merging
- Teams may branch/merge multiple times per day

Session Objectives

- Understand how commits, trees, and blobs relate to branches
- Create and switch between branches
- Perform a fast-forward merge
- Perform a three-way merge
- Follow a real-world branch-hotfix-merge workflow

Recap: Commits Are Snapshots

- Git stores data as snapshots, not diffs
- Each commit object contains:
  - Pointer to the staged snapshot (tree)
  - Author name and email
  - Commit message
  - Pointer(s) to parent commit(s)

Parent Commit Rules

- Zero parents $\rightarrow$ initial commit
- One parent $\rightarrow$ normal commit
- Multiple parents $\rightarrow$ merge commit

What a Commit Actually Stores

- `git add README test.rb LICENSE`
- `git commit -m 'Initial commit'`
- Result: 3 blobs (file contents) + 1 tree (directory structure) + 1 commit object

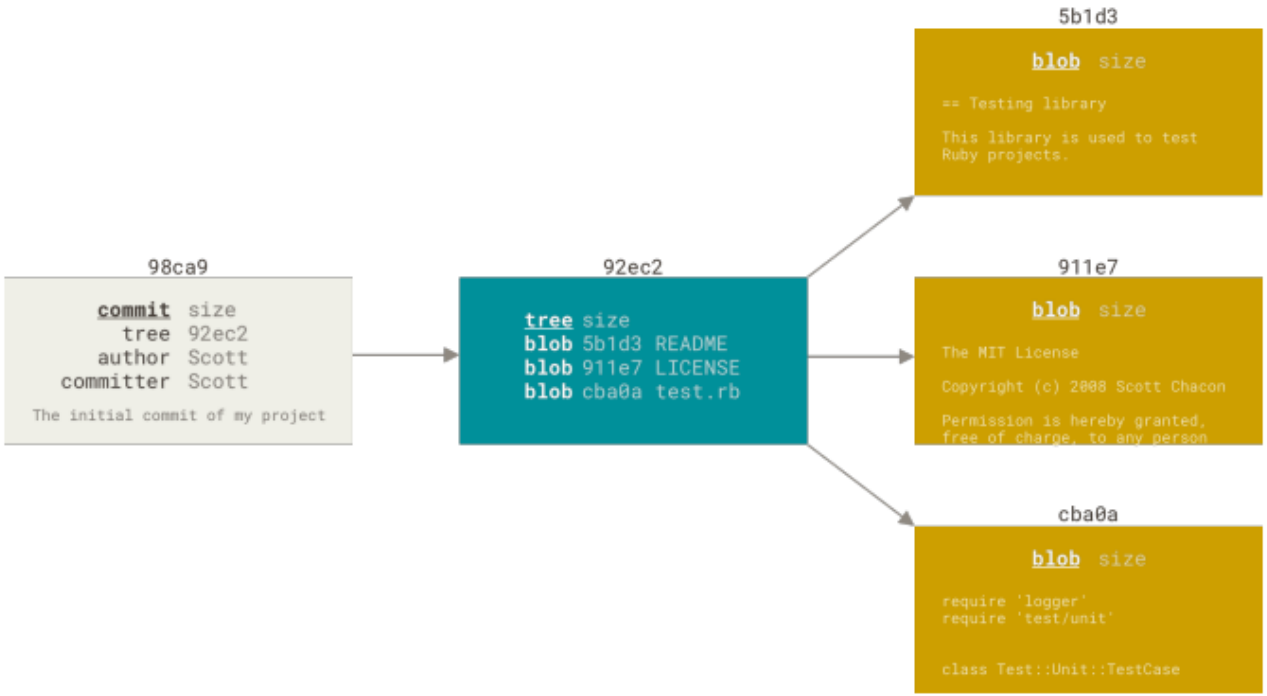


Figure 9. A commit and its tree

What a Commit Actually Stores

- If you make some changes and commit again, the next commit stores a pointer to the commit that came immediately before it.

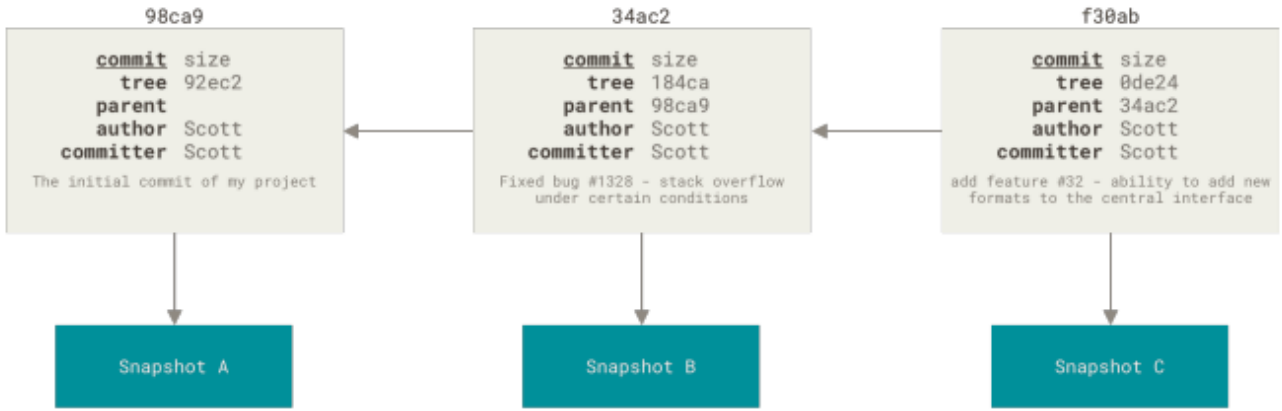


Figure 10. Commits and their parents

What a Branch Really Is

- A branch is simply a movable pointer to a commit
- Default branch name in Git itself: `master`
- GitHub and others now default to `main`
- Creating a branch just creates a new pointer — extremely cheap

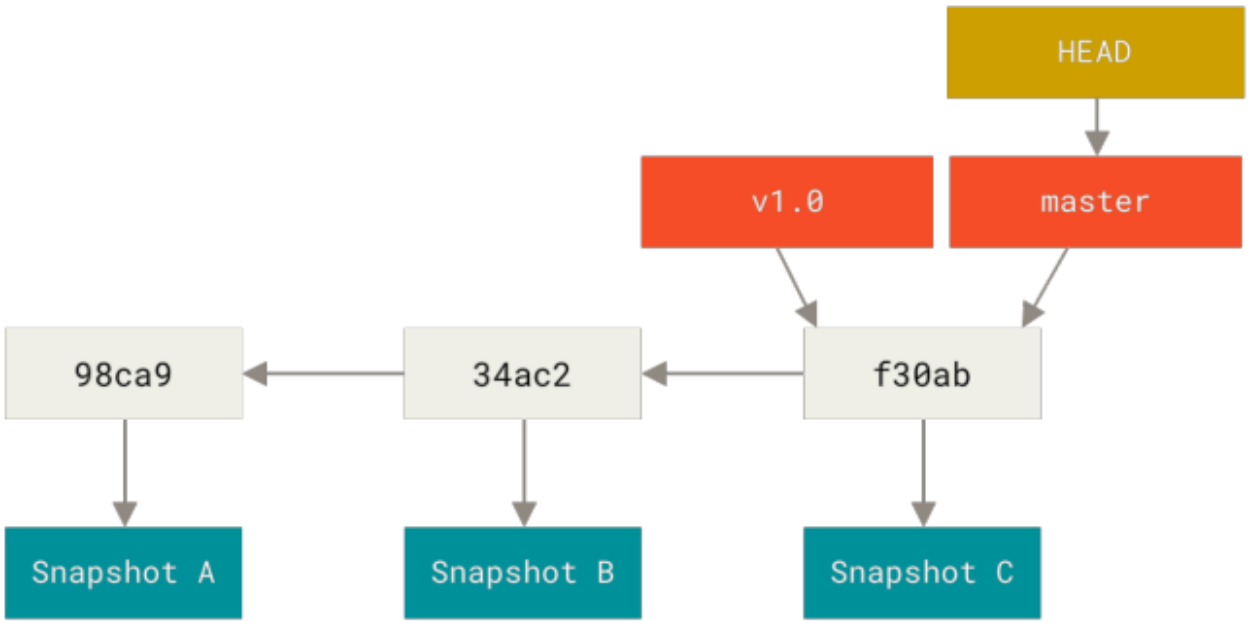


Figure 11. A branch and its commit history